El esquema de Shamir para compartir secretos

Fundamentos, algoritmos, librerías en Python y aplicaciones a claves de clave pública

Dr. José de Jesús Ángel Ángel
Facultad de Ingeniería, Universidad Anáhuac México

21 de octubre de 2026 (versión revisada)

Resumen

El esquema de Shamir (1979) permite dividir un secreto S en n partes de modo que cualesquiera k de ellas lo reconstruyen y cualesquiera $k-1$ no revelan *nada* sobre él. La idea es geométrica y elemental: un polinomio de grado $k-1$ queda determinado por k puntos. Este documento presenta los fundamentos algebraicos, la prueba de seguridad perfecta, un ejemplo numérico completo, los algoritmos de reparto y reconstrucción en pseudocódigo, una implementación de referencia en Python y una revisión de cuatro librerías mantenidas (PyCryptodome, `shamirs`, `sslib` y `shamir-mnemonic`). Después revisa aplicaciones reales en operación (la raíz de DNSSEC, HashiCorp Vault, billeteras de criptomonedas) y dedica una sección completa a proteger claves privadas de criptografía de clave pública (Ed25519, ECDSA sobre secp256k1 y RSA), incluyendo la verificación de partes de Feldman, la interpolación "en el exponente" y una firma de Schnorr de umbral en la que la clave privada nunca se reconstruye. Cierra con las limitaciones del esquema.

Índice general

1 El problema de compartir un secreto **2**

2 Fundamentos algebraicos **3**
2.1 Campos finitos . . . 3
2.2 Interpolación de Lagrange . . . 3

3 El esquema de Shamir **3**
3.1 Descripción . . . 3
3.2 Corrección y seguridad perfecta . . . 4
3.3 Propiedades útiles . . . 4

4 Ejemplo numérico completo **5**

5 Algoritmos **5**

6 Implementación de referencia en Python **6**

7 Librerías en Python **8**
7.1 PyCryptodome: `Crypto.Protocol.SecretSharing` . . . 9
7.2 `shamirs`: implementación mínima sobre $\mathbb{F}_p$ . . . 9
7.3 `sslib`: secretos de longitud arbitraria . . . 10
7.4 `shamir-mnemonic`: el estándar SLIP-39 . . . 10

8 Aplicaciones reales **11**
8.1 La raíz de DNSSEC . . . 11
8.2 HashiCorp Vault . . . 12
8.3 Billeteras de criptomonedas . . . 12
8.4 Firmas de umbral y estandarización . . . 12

9 Claves privadas de criptografía de clave pública **12**
9.1 Respaldo de una clave Ed25519 . . . 13
9.2 ECDSA sobre secp256k1: repartir en el campo del grupo . . . 13
9.3 Firma de Schnorr de umbral: la clave nunca se reconstruye . . . 15
9.4 RSA: el patrón híbrido . . . 16
9.5 Distribuir las partes cifradas con la clave pública de cada custodio . . . 17

10 Limitaciones y extensiones **17**

Lista de algoritmos

1 Reparto de Shamir (k, n) . . . 6
2 Reconstrucción por interpolación de Lagrange en $x = 0$ . . . 7
3 Inverso modular por el algoritmo extendido de Euclides . . . 8

1 El problema de compartir un secreto

Supongamos que una organización protege una clave maestra (la llave de una autoridad certificadora, la semilla de una billetera de criptomonedas, la clave de descifrado de un respaldo). Si la guarda una sola persona, hay un punto único de falla: esa persona puede perderla, ser coaccionada o actuar de mala fe. Si se copia íntegra a varias personas, el riesgo de filtración se multiplica.

Lo que se desea es un *control de umbral*: que ninguna coalición pequeña pueda usar la clave, pero que cualquier coalición suficientemente grande sí pueda, aun si algunos custodios no están disponibles.

Definición 1.1. (Esquema de umbral (k, n))

Un *esquema de umbral* (k, n), con $1 \le k \le n$, es un par de algoritmos

- **Reparto:** a partir de un secreto S produce partes $s_1, \dots, s_n$, una para cada participante;
- **Reconstrucción:** a partir de cualesquiera k partes distintas recupera S;

tales que cualquier conjunto de a lo más $k - 1$ partes no proporciona información sobre S. Si esta última condición se cumple en sentido de teoría de la información (no sólo computacional), el esquema es *perfecto*.

Adi Shamir [1] y, de manera independiente, George Blakley [2] publicaron en 1979 las primeras construcciones. La de Blakley usa intersecciones de hiperplanos; la de Shamir, interpolación polinomial, y es la que se ha impuesto en la práctica por su sencillez y porque cada parte tiene el mismo tamaño que el secreto.

2 Fundamentos algebraicos

2.1 Campos finitos

Trabajamos en un campo finito $\mathbb{F}_q$. En la versión clásica $q = p$ es primo y $\mathbb{F}_p = \mathbb{Z}_p = \{0, 1, \dots, p-1\}$ con suma y producto módulo p. Todo elemento no nulo tiene inverso multiplicativo, calculable con el algoritmo extendido de Euclides o con el pequeño teorema de Fermat, $a^{-1} \equiv a^{p-2} \pmod{p}$.

En implementaciones orientadas a bytes es común usar $\mathbb{F}_{2^8}$ (cada byte del secreto se reparte por separado) o $\mathbb{F}_{2^{128}}$ (como hace PyCryptodome). La teoría es idéntica: sólo cambia la aritmética del campo.

2.2 Interpolación de Lagrange

Teorema 2.1. (Existencia y unicidad)

Sean $x_1, \dots, x_k \in \mathbb{F}$ distintos y $y_1, \dots, y_k \in \mathbb{F}$ arbitrarios. Existe un único polinomio $f \in \mathbb{F}[x]$ de grado a lo más $k-1$ tal que $f(x_i) = y_i$ para $i = 1, \dots, k$. Este polinomio es

$$f(x) = \sum_{j=1}^{k} y_j \, \ell_j(x), \qquad \ell_j(x) = \prod_{\substack{m=1 \\ m \neq j}}^{k} \frac{x - x_m}{x_j - x_m}. \tag{1}$$

Prueba. Cada ℓ_j tiene grado $k-1$ y satisface $\ell_j(x_i) = \delta_{ij}$, de modo que f interpola los datos. Si g fuera otro interpolante de grado $\leq k-1$, entonces $f - g$ tendría grado $\leq k-1$ y k raíces distintas en un campo, luego $f - g = 0$. □

Para reconstruir el secreto sólo necesitamos $f(0)$. Evaluando (1) en $x = 0$:

$$S = f(0) = \sum_{j=1}^{k} y_j \, \lambda_j, \qquad \lambda_j = \prod_{\substack{m=1 \\ m \neq j}}^{k} \frac{x_m}{x_m - x_j}. \tag{2}$$

Los λ_j se llaman *coeficientes de Lagrange* y dependen únicamente de las abscisas, no de los valores de las partes.

3 El esquema de Shamir

3.1 Descripción

Parámetros públicos. Un primo $p > \max(S, n)$ y abscisas distintas y no nulas $x_1, \dots, x_n \in \mathbb{F}_p$; usualmente $x_i = i$.

Reparto (lo ejecuta un *repartidor* de confianza).

1. Fijar $a_0 = S$.
2. Elegir $a_1, \dots, a_{k-1}$ uniformemente al azar en $\mathbb{F}_p$.
3. Formar $f(x) = a_0 + a_1 x + \cdots + a_{k-1} x^{k-1}$.
4. Entregar al participante i la parte $(x_i,\, y_i)$ con $y_i = f(x_i)$.
5. Borrar f y los coeficientes.

Reconstrucción. Cualesquiera k participantes juntan sus pares y calculan $S = f(0)$ con (2).

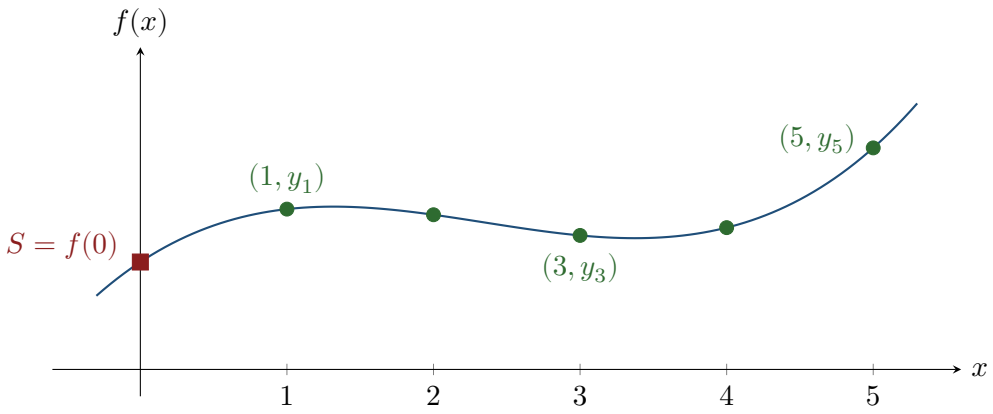


Figura 1: Intuición geométrica con $k = 4$ (polinomio cúbico) y $n = 5$, dibujada sobre $\mathbb{R}$ sólo con fines ilustrativos. Cuatro puntos verdes cualesquiera determinan la curva y, por tanto, su ordenada al origen S. Con tres puntos hay infinitas cúbicas que los atraviesan, una por cada valor posible de S. En el esquema real la aritmética es módulo p y la "curva" es un conjunto discreto.

3.2 Corrección y seguridad perfecta

La corrección es inmediata del Teorema 2.1: k puntos distintos determinan f y, en particular, $f(0) = S$.

Teorema 3.1. (Seguridad perfecta)

Sea $T \subset \{1, \dots, n\}$ con $|T| = k - 1$. Para todo valor candidato $S' \in \mathbb{F}_p$ existe exactamente un polinomio de grado $\leq k - 1$ que pasa por los $k - 1$ puntos $\{(x_i, y_i)\}_{i \in T}$ y por $(0, S')$. En consecuencia, si los coeficientes $a_1, \dots, a_{k-1}$ son uniformes e independientes, la distribución de las partes $\{y_i\}_{i \in T}$ es la misma para cualquier secreto, es decir,

$$\Pr[S = s \mid (y_i)_{i \in T}] = \Pr[S = s] \qquad \text{para todo } s.$$

Prueba. Los k puntos $\{(x_i, y_i)\}_{i \in T} \cup \{(0, S')\}$ tienen abscisas distintas (porque $x_i \neq 0$), así que por el Teorema 2.1 hay exactamente un interpolante. Visto de otra forma: para S fijo, la función $(a_1, \dots, a_{k-1}) \mapsto (y_i)_{i \in T}$ es lineal y su matriz es una Vandermonde (escalada) de $(k-1) \times (k-1)$ con nodos distintos y no nulos, por lo tanto invertible. Una biyección afín envía la distribución uniforme en $\mathbb{F}_p^{k-1}$ en la uniforme en $\mathbb{F}_p^{k-1}$, independientemente de S. □

Por qué $x_i \neq 0$

Si algún participante recibiera $x_i = 0$, su parte sería $f(0) = S$: tendría el secreto completo. Por eso las abscisas se toman en $\{1, \dots, n\}$ y se exige $p > n$ para que sean distintas módulo p.

3.3 Propiedades útiles

- **Eficiencia de espacio.** Cada parte tiene el tamaño del secreto (más el índice x_i, que no es confidencial).

- **Extensibilidad.** Se pueden emitir nuevas partes evaluando f en nuevas abscisas, sin tocar las existentes (mientras se conserve f).
- **Renovación.** Sumando a cada parte la evaluación de un polinomio aleatorio g con $g(0) = 0$ se obtienen partes nuevas del mismo secreto; las viejas quedan inservibles combinadas con las nuevas. Es la base de la *compartición proactiva*.
- **Jerarquías.** Dar más de una parte a un participante le asigna más peso en el umbral.
- **Linealidad.** Si (x_i, y_i) y (x_i, z_i) son partes de S y T con el mismo umbral, entonces $(x_i, y_i + z_i)$ es una parte de $S + T$. Esta propiedad homomórfica es el punto de partida del cómputo seguro multipartito.

4 Ejemplo numérico completo

Ejemplo: esquema $(3, 5)$ sobre $\mathbb{F}_{31}$

Tomemos $p = 31$, secreto $S = 13$, umbral $k = 3$ y $n = 5$ participantes. El repartidor elige al azar $a_1 = 7$ y $a_2 = 4$, de modo que

$$f(x) = 13 + 7x + 4x^2 \pmod{31}.$$

Reparto. Las partes son

x_i	1	2	3	4	5
$f(x_i)$ sin reducir	24	43	70	105	148
$y_i = f(x_i) \bmod 31$	**24**	**12**	**8**	**12**	**24**

Reconstrucción con los participantes 1, 2 y 3. Los coeficientes de Lagrange en $x = 0$ son

$$\lambda_1 = \frac{2}{2-1} \cdot \frac{3}{3-1} = 3, \qquad \lambda_2 = \frac{1}{1-2} \cdot \frac{3}{3-2} = -3, \qquad \lambda_3 = \frac{1}{1-3} \cdot \frac{2}{2-3} = 1.$$

Entonces

$$S = 24 \cdot 3 + 12 \cdot (-3) + 8 \cdot 1 = 72 - 36 + 8 = 44 \equiv 13 \pmod{31}. \checkmark$$

Dos partes no bastan. Con sólo $(1, 24)$ y $(2, 12)$, para cada $S' \in \{0, \dots, 30\}$ existe una única parábola módulo 31 que pasa por $(0, S')$, $(1, 24)$ y $(2, 12)$; los 31 secretos son igualmente consistentes con lo observado.

5 Algoritmos

Presentamos el esquema en tres rutinas. La evaluación del polinomio usa el método de Horner ($k - 1$ multiplicaciones y sumas) y la reconstrucción evalúa (2) con una sola inversión modular por término.

Complejidad. El reparto cuesta $O(nk)$ operaciones en el campo y la reconstrucción con k partes, $O(k^2)$ multiplicaciones más k inversiones. Si se reconstruyen muchos secretos con el mismo conjunto

Algoritmo 1 Reparto de Shamir (k, n)

Entrada: secreto $S \in \mathbb{F}_p$; umbral k; número de partes n, con $2 \le k \le n < p$
Salida: lista de partes $\{(x_i, y_i)\}_{i=1}^{n}$

función *Repartir*(S, k, n, p)
  si $\neg(0 \le S < p)$ **o** $\neg(2 \le k \le n < p)$ **entonces**
    error: parámetros inválidos
  fin si
  $a_0 \leftarrow S$
  para $j \leftarrow 1$ **a** $k-1$ **hacer**
    $a_j \leftarrow$ *AleatorioUniforme*$(\mathbb{F}_p)$ ▷ generador criptográficamente seguro
  fin para
  $P \leftarrow [\,]$
  para $i \leftarrow 1$ **a** n **hacer**
    $x_i \leftarrow i$
    $y_i \leftarrow$ *Horner*$((a_0, \dots, a_{k-1}),\, x_i,\, p)$
    agregar (x_i, y_i) a P
  fin para
  borrar de memoria $a_1, \dots, a_{k-1}$
  devolver P
fin función

función *Horner*$((a_0, \dots, a_d),\, x,\, p)$
  $y \leftarrow 0$
  para $j \leftarrow d$ **bajando a** 0 **hacer**
    $y \leftarrow (y \cdot x + a_j) \bmod p$
  fin para
  devolver y
fin función

de abscisas, los λ_j se precalculan una vez y cada reconstrucción posterior es un producto punto de costo $O(k)$.

Sobre el umbral en la reconstrucción. La fórmula de Lagrange con $t < k$ partes *no falla*: devuelve el valor en 0 de otro polinomio de grado $t-1$, que no tiene relación con S. Por eso el algoritmo debe conocer k y rechazar conjuntos pequeños. Con $t > k$ partes correctas el resultado sigue siendo S, pues el único interpolante de grado $\le t-1$ es el propio f.

Secretos largos. Cuando S es una cadena de bytes larga hay dos opciones habituales: (a) repartir byte a byte sobre $\mathbb{F}_{2^8}$, con un polinomio independiente por byte, o (b) cifrar los datos con una clave simétrica (AES) y repartir sólo la clave. La segunda es la recomendada para archivos.

6 Implementación de referencia en Python

La siguiente implementación usa sólo la biblioteca estándar (Python $\ge$ 3.8, por `pow(den, -1, p)`) y trabaja sobre el primo de Mersenne $M_{127} = 2^{127} - 1$, suficiente para secretos de hasta 126 bits. Es didáctica: sigue los Algoritmos 1 y 2. Por simplicidad, `reconstruir` no recibe k: es

Algoritmo 2 Reconstrucción por interpolación de Lagrange en $x = 0$

Entrada: t partes $\{(x_j, y_j)\}_{j=1}^{t}$; umbral k; primo p
Salida: el secreto $S = f(0)$

función *Reconstruir*($\{(x_j, y_j)\}$, k, p)
  si $t < k$ **entonces**
    error: partes insuficientes ▷ con $t < k$ el resultado sería basura
  fin si
  si algún $x_j = 0$ **o** existen $j \neq m$ con $x_j = x_m$ **entonces**
    error: abscisas nulas o repetidas
  fin si
  $S \leftarrow 0$
  para $j \leftarrow 1$ **a** t **hacer**
    $num \leftarrow 1$; $den \leftarrow 1$
    para $m \leftarrow 1$ **a** t, $m \neq j$ **hacer**
      $num \leftarrow num \cdot (-x_m) \bmod p$
      $den \leftarrow den \cdot (x_j - x_m) \bmod p$
    fin para
    $\lambda_j \leftarrow num \cdot InversoMod(den, p) \bmod p$
    $S \leftarrow (S + y_j\, \lambda_j) \bmod p$
  fin para
  devolver S
fin función

responsabilidad de quien la llama presentar al menos k partes. El parámetro `p` permite trabajar con primos mayores, como se hace en la Sección 9.

Listing 1: `shamir_ref.py`: implementación directa de los algoritmos

```python
import secrets

P = 2**127 - 1          # primo de Mersenne M127

def _eval(coefs, x, p):
    """Evalúa el polinomio por el método de Horner."""
    y = 0
    for a in reversed(coefs):
        y = (y * x + a) % p
    return y

def repartir(s, k, n, p=P):
    """Divide el secreto s en n partes con umbral k."""
    if not (0 <= s < p and 1 < k <= n < p):
        raise ValueError("parámetros inválidos")
    coefs = [s] + [secrets.randbelow(p) for _ in range(k - 1)]
    return [(i, _eval(coefs, i, p)) for i in range(1, n + 1)]

def reconstruir(partes, p=P):
    """Interpolación de Lagrange evaluada en x = 0."""
    xs = [x for x, _ in partes]
    if 0 in xs or len(set(xs)) != len(xs):
        raise ValueError("abscisas nulas o repetidas")
```

Algoritmo 3 Inverso modular por el algoritmo extendido de Euclides

Entrada: $a \in \{1, \dots, p-1\}$, p primo
Salida: $a^{-1} \bmod p$

función *InversoMod*(a, p)
  $(r_0, r_1) \leftarrow (p,\ a \bmod p)$; $\quad (t_0, t_1) \leftarrow (0, 1)$
  mientras $r_1 \neq 0$ **hacer**
    $q \leftarrow \lfloor r_0 / r_1 \rfloor$
    $(r_0, r_1) \leftarrow (r_1,\ r_0 - q\, r_1)$
    $(t_0, t_1) \leftarrow (t_1,\ t_0 - q\, t_1)$
  fin mientras
  si $r_0 \neq 1$ **entonces**
    error: a no es invertible
  fin si
  devolver $t_0 \bmod p$
fin función

```
    s = 0
    for j, (xj, yj) in enumerate(partes):
        num, den = 1, 1
        for m, (xm, _) in enumerate(partes):
            if m != j:
                num = num * (-xm) % p
                den = den * (xj - xm) % p
        s = (s + yj * num * pow(den, -1, p)) % p
    return s

if __name__ == "__main__":
    S = 20261021
    partes = repartir(S, k=3, n=5)
    assert reconstruir(partes[:3]) == S
    assert reconstruir([partes[0], partes[2], partes[4]]) == S
    print("OK:", reconstruir(partes[1:4]))
```

Use `secrets`, nunca `random`

El módulo `random` de Python usa Mersenne Twister, que *no* es criptográficamente seguro: tras observar suficientes salidas su estado interno puede reconstruirse, y con él los coeficientes a_j. La seguridad perfecta del Teorema 3.1 supone coeficientes uniformes e impredecibles; por eso la implementación usa `secrets.randbelow`.

7 Librerías en Python

Las cuatro librerías siguientes se instalaron y probaron con los fragmentos que aparecen en esta sección (versiones indicadas en el Cuadro 1).

Tabla 1: Comparación de librerías de Python para el esquema de Shamir.

Paquete (`pip`)	**Versión**	**Campo**	**Tipo de secreto**	**Uso típico**
`pycryptodome`	4.1.0	$\mathbb{F}_{2^{128}}$	exactamente 16 bytes	Repartir una clave AES-128 que cifra un archivo
`shamirs`	4.0.0	$\mathbb{F}_p$, por omisión $p = 2^{127} - 1$	entero no negativo	Docencia y cómputo multipartito (suma de partes, producto por escalar)
`sslib`	0.2.0	$\mathbb{F}_p$, p elegido según el tamaño	bytes de longitud arbitraria	Repartir contraseñas o archivos pequeños
`shamir-mnemonic`	0.3.0	$\mathbb{F}_{2^8}$ (SLIP-39)	semilla de 16 a 32 bytes	Respaldo de billeteras en frases de palabras, con grupos

7.1 PyCryptodome: `Crypto.Protocol.SecretSharing`

Es la opción más robusta para uso general por formar parte de una biblioteca criptográfica madura y de uso muy extendido. Su clase `Shamir` reparte secretos de exactamente 16 bytes (una clave AES-128) trabajando en $\mathbb{F}_{2^{128}}$ [19]. El argumento opcional `ssss=True` produce partes compatibles con la herramienta de línea de comandos `ssss`.

Listing 2: Reparto de una clave AES con PyCryptodome

```
from Crypto.Protocol.SecretSharing import Shamir
from Crypto.Random import get_random_bytes

clave = get_random_bytes(16)                  # p. ej. una clave AES-128
partes = Shamir.split(3, 5, clave)            # lista de (índice, 16 bytes)
assert Shamir.combine(partes[1:4]) == clave   # cualesquiera 3 bastan
```

Para secretos mayores, el patrón recomendado por la propia documentación es cifrar los datos con AES en un modo autenticado (EAX o GCM) y repartir sólo la clave. Además, la etiqueta de autenticación del cifrado detecta, al descifrar, si alguna parte presentada era incorrecta.

7.2 `shamirs`: implementación mínima sobre $\mathbb{F}_p$

Es una implementación en Python puro, pequeña y legible, que trabaja con enteros y obtiene sus coeficientes con `secrets.token_bytes` y muestreo por rechazo [20]. Su interés adicional es que expone la linealidad del esquema mediante `add` (suma de partes de dos secretos) y `mul` (producto de las partes por un escalar público), útiles para ilustrar el cómputo seguro multipartito. Multiplicar *dos* secretos compartidos no es tan simple: el producto de las partes es un punto de un polinomio de grado $2(k-1)$ y exige un protocolo de reducción de grado.

Listing 3: Uso básico y suma homomórfica con `shamirs`

```
import shamirs

ps = shamirs.shares(123456789, quantity=5, threshold=3)
assert shamirs.interpolate(ps[2:]) == 123456789

# Suma de dos secretos sin revelarlos: se suman las partes
```

```python
a = shamirs.shares(10, quantity=3, threshold=3)
b = shamirs.shares(32, quantity=3, threshold=3)
c = [shamirs.add(x, y) for x, y in zip(a, b)]
assert shamirs.interpolate(c) == 42
```

El módulo por omisión es $2^{127} - 1$ y puede cambiarse con el parámetro `modulus`.

7.3 `sslib`: secretos de longitud arbitraria

Reparte cualquier cadena de bytes eligiendo un primo apenas mayor que el secreto. Devuelve un diccionario con el umbral, el primo y la lista de partes; la función `to_base64` facilita guardarlas como texto. Toma su aleatoriedad del sistema operativo (`/dev/random` u `os.urandom`) y rechaza la reconstrucción si se presentan menos partes que el umbral.

Listing 4: Reparto de una contraseña con `sslib`

```python
from sslib import shamir

datos = shamir.split_secret("contraseña maestra".encode(), 3, 5)
# datos = {"required_shares": 3, "prime_mod": ..., "shares": [...]}
sub = {"required_shares": datos["required_shares"],
       "prime_mod": datos["prime_mod"],
       "shares": datos["shares"][:3]}
assert shamir.recover_secret(sub).decode() == "contraseña maestra"
```

Detalle de implementación de `sslib`

Los coeficientes se obtienen reduciendo módulo p un bloque aleatorio de bytes (`int(bytes) % p`), lo que en principio introduce un sesgo. Al medirlo para los primos que la librería elige, la distancia estadística a la distribución uniforme es de 2^{-17} para secretos de 2 bytes, pero de 2^{-61} o menos para secretos de 5 bytes en adelante: despreciable en la práctica. El muestreo por rechazo (`secrets.randbelow`, como en `shamirs` y en la implementación de referencia) elimina el sesgo por completo.

7.4 `shamir-mnemonic`: el estándar SLIP-39

Es la implementación de referencia, mantenida por Trezor (SatoshiLabs), del estándar SLIP-39 [17, 18]. Cada parte se codifica como una frase de 20 o 33 palabras con suma de verificación, de manera que un error de transcripción se detecta. Admite un esquema de *dos niveles*: el secreto se reparte entre grupos con un umbral de grupos, y dentro de cada grupo con su propio umbral. Opcionalmente cifra la semilla con una frase de contraseña.

Listing 5: Esquema simple y de dos niveles con SLIP-39

```python
import shamir_mnemonic as sm

semilla = bytes.fromhex("bb54aac4b89dc868ba37d9cc21b2cece")

# Un solo grupo con umbral 3 de 5
grupos = sm.generate_mnemonics(1, [(3, 5)], semilla, passphrase=b"")
frases = grupos[0]                          # cinco frases de 20 palabras
assert sm.combine_mnemonics(frases[:3]) == semilla

```

```python
# Dos niveles: se requieren 2 de 3 grupos
#   grupo 0: 1 de 1 (yo), grupo 1: 2 de 3 (familia), grupo 2: 3 de 5 (amigos)
g = sm.generate_mnemonics(2, [(1, 1), (2, 3), (3, 5)], semilla)
assert sm.combine_mnemonics(g[0] + g[1][:2]) == semilla
```

Instalando el extra `shamir-mnemonic[cli]` se obtiene la orden `shamir`:

```
$ shamir create 3of5     # crea 5 frases con umbral 3
$ shamir recover         # recombina las frases de forma interactiva
```

Sus autores advierten que esa herramienta de línea de comandos es una referencia de prueba y no debe usarse para secretos reales.

Bibliotecas antiguas

En muchos tutoriales aparece `secretsharing` (`SecretSharer`). Es un proyecto antiguo cuyos ejemplos están escritos para Python 2; para trabajo nuevo conviene preferir las opciones de esta sección.

8 Aplicaciones reales

El esquema de Shamir no es una curiosidad académica: protege algunas de las claves más valiosas de Internet y de los sistemas financieros. El Cuadro 2 resume los casos documentados que se describen en esta sección.

Tabla 2: Despliegues del esquema de Shamir y sus variantes.

Sistema	Qué se reparte	Umbral	Variante
Raíz de DNSSEC (ICANN/IANA)	Clave maestra de almacenamiento que cifra los respaldos de la KSK raíz	5 de 7	Partes en poder de personas de distintos países
HashiCorp Vault	Clave de desellado (*unseal key*) o clave de recuperación	3 de 5 (por omisión)	Partes cifradas con PGP de cada operador
Trezor (Shamir Backup)	Semilla maestra de la billetera	configurable	SLIP-39, dos niveles, frases de palabras
Ledger Recover	Frase de recuperación de la billetera	2 de 3	VSS de Pedersen entre tres empresas
Firmas de umbral (FROST)	Clave privada de firma Schnorr	configurable	La clave nunca se reconstruye

8.1 La raíz de DNSSEC

DNSSEC firma la zona raíz del DNS con una clave de firma de claves (KSK) que reside en módulos de seguridad en hardware (HSM) en dos instalaciones de Estados Unidos. Su política de gestión de claves [12] establece dos controles de umbral distintos:

- Para *activar* la KSK en una ceremonia se requieren tres de siete tarjetas inteligentes en poder de los oficiales criptográficos (*Crypto Officers*), que deben pertenecer a organizaciones ajenas a ICANN y Verisign.
- Para *recuperar* las claves si se perdieran todos los HSM, la clave maestra de almacenamiento (SMK), que cifra los respaldos de la KSK, se exporta en partes secretas con umbral 5 de 7. La política exige que ninguna persona u organización tenga jamás la SMK completa.

Este segundo mecanismo, en vigor desde que la raíz se firmó en 2010, es quizá el ejemplo más citado de compartición de secretos en la infraestructura de Internet [13]. Nótese el diseño: no se reparte la KSK, sino la clave que cifra sus respaldos, exactamente el patrón híbrido de la Sección 9.4.

8.2 HashiCorp Vault

Vault es un gestor de secretos muy usado en centros de datos. Al arrancar, sus datos están cifrados y el servidor está "sellado". Con la configuración por omisión, la clave de desellado se reparte con Shamir; los operadores introducen sus partes una por una hasta alcanzar el umbral, la clave de desellado se reconstruye y descifra la clave raíz [14]. Los valores por omisión de `vault operator init` son 5 partes con umbral 3, y con la opción `-pgp-keys` cada parte sale cifrada con la clave pública PGP de su custodio [15]: nadie, ni siquiera quien inicializa el servidor, ve las partes en claro. Cuando el desellado se delega a un HSM o a un servicio de gestión de claves en la nube, Vault reparte en su lugar una clave de recuperación, que autoriza operaciones de quórum pero no descifra los datos.

8.3 Billeteras de criptomonedas

En una billetera, perder la semilla equivale a perder los fondos y filtrarla equivale a regalarlos. Dos fabricantes de billeteras en hardware usan el esquema de Shamir de maneras opuestas:

- **Trezor** implementa SLIP-39 (Sección 7): el usuario reparte su semilla entre personas o lugares que él elige, con uno o dos niveles de umbral, y cada parte es una frase de palabras con suma de verificación.
- **Ledger Recover** es un servicio opcional que reparte la frase de recuperación en tres partes con umbral 2 de 3, en poder de Coincover, EscrowTech y la propia Ledger, usando la variante verificable de Pedersen para que el orquestador compruebe que cada proveedor recibió una parte correcta [16]. Su lanzamiento en 2023 generó polémica precisamente porque las partes salen del dispositivo: el modelo de confianza pasa a depender de que dos de las tres empresas no se coludan.

8.4 Firmas de umbral y estandarización

La aplicación con más actividad hoy son las firmas de umbral, en las que los custodios firman con sus partes sin reconstruir nunca la clave (Sección 9.3). El protocolo FROST para firmas de Schnorr quedó estandarizado en el RFC 9591 [6], y la Zcash Foundation mantiene implementaciones en Rust para Ed25519, secp256k1 y otras curvas [11]. Por su parte, el NIST publicó el 20 de enero de 2026 el informe IR 8214C, su primera convocatoria a esquemas multipartitos de umbral, que abarca firmas, cifrado, generación de claves y otras primitivas [10].

9 Claves privadas de criptografía de clave pública

En criptografía de clave pública el secreto valioso es la clave privada: la de una autoridad certificadora, la de firma de código de un fabricante, la de un servidor o la que controla una billetera. La clave pública puede publicarse; la privada debe sobrevivir a pérdidas y no quedar en manos de una sola persona. El esquema de Shamir encaja de forma natural, pero la manera correcta de aplicarlo depende de la estructura algebraica de la clave.

Principio de diseño

Si la clave privada es un elemento de un campo primo $\mathbb{F}_q$ (el caso de las curvas elípticas, donde la clave es un escalar módulo el orden primo q del grupo), conviene repartirla *en ese mismo campo*: además del respaldo, se obtienen verificación de partes, cálculo de la clave pública a partir de las partes y firmas de umbral. Si la clave no vive en un campo (RSA) o tiene un formato propio (la semilla de Ed25519), lo práctico es repartirla como cadena de bytes en un campo suficientemente grande o, mejor aún, cifrarla y repartir sólo la clave de cifrado.

9.1 Respaldo de una clave Ed25519

Una clave privada Ed25519 (RFC 8032 [9]) es una semilla de 32 bytes. Basta un primo mayor que 2^{256}; usamos el primo de Mersenne $M_{521} = 2^{521} - 1$ con la implementación de referencia de la Sección 6. El ejemplo usa la librería `cryptography` para generar la clave, firmar y verificar.

Listing 6: Reparto y recuperación de una clave Ed25519

```python
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
from cryptography.hazmat.primitives import serialization as ser
from shamir_ref import repartir, reconstruir

P521 = 2**521 - 1                       # primo de Mersenne, mayor que 2^256

sk = Ed25519PrivateKey.generate()
pk = sk.public_key()
semilla = sk.private_bytes(ser.Encoding.Raw, ser.PrivateFormat.Raw,
                           ser.NoEncryption())          # 32 bytes
partes = repartir(int.from_bytes(semilla, "big"), k=3, n=5, p=P521)
del sk, semilla                         # el original se destruye

# ... tiempo después, tres custodios se reúnen
s = reconstruir([partes[0], partes[3], partes[4]], p=P521)
sk2 = Ed25519PrivateKey.from_private_bytes(s.to_bytes(32, "big"))
firma = sk2.sign(b"acta de la sesion")
pk.verify(firma, b"acta de la sesion")  # lanza excepción si falla
```

La semilla de Ed25519 no es el escalar

En Ed25519 el escalar secreto d se *deriva* de la semilla aplicando SHA-512 y ajustando algunos bits. Repartir la semilla sirve para respaldo, pero no permite las construcciones algebraicas de las secciones siguientes (las partes de la semilla no son partes de d). Por eso las firmas de umbral sobre Ed25519, como FROST, reparten directamente el escalar y producen claves que no tienen semilla en el formato de RFC 8032.

9.2 ECDSA sobre secp256k1: repartir en el campo del grupo

Sea E una curva elíptica con un punto generador G de orden primo q. La clave privada es $d \in \{1, \ldots, q-1\}$ y la pública es $Q = dG$. Si repartimos d con un polinomio f sobre $\mathbb{F}_q$, con $f(0) = d$, aparecen tres propiedades que no existen cuando se reparte una cadena de bytes cualquiera.

1. Verificación de Feldman con costo nulo de privacidad. El repartidor publica los compromisos $C_j = a_j G$ de los coeficientes. Cada custodio comprueba su parte sin conocer las demás:

$$y_i G \stackrel{?}{=} \sum_{j=0}^{k-1} x_i^j \, C_j. \tag{3}$$

La igualdad se cumple porque $y_i G = f(x_i)G = \sum_j a_j x_i^j G$. En general, la VSS de Feldman sólo oculta el secreto computacionalmente, porque revela $C_0 = SG$. Aquí eso no cuesta nada: $C_0 = dG = Q$ *es* la clave pública, que de todos modos es pública.

2. Interpolación en el exponente. Cada custodio puede publicar su *clave pública parcial* $Q_i = y_i G$. Como la multiplicación escalar es lineal, para cualquier conjunto T de k custodios,

$$\sum_{j \in T} \lambda_j \, Q_j = \Big(\sum_{j \in T} \lambda_j \, y_j\Big) G = dG = Q, \tag{4}$$

con los mismos coeficientes de Lagrange λ_j de (2), ahora calculados módulo q. Se obtiene la clave pública sin que d exista en ningún lugar. Esta es la idea que hace posibles las firmas de umbral.

3. Firma de umbral. Véase la Sección 9.3.

El siguiente programa usa la librería `ecdsa` (Python puro, con aritmética de puntos) y comprueba las tres primeras afirmaciones antes de firmar con la clave reconstruida.

Listing 7: `pk_secp256k1.py`: reparto en $\mathbb{F}_q$, verificación de Feldman e interpolación en el exponente

```python
import secrets
from ecdsa import SECP256k1, SigningKey, VerifyingKey

G, n = SECP256k1.generator, SECP256k1.order   # n es primo

def lagrange0(xs, n):
    """Coeficientes de Lagrange en x = 0 sobre Z_n."""
    lam = []
    for xj in xs:
        num = den = 1
        for xm in xs:
            if xm != xj:
                num = num * xm % n
                den = den * (xm - xj) % n
        lam.append(num * pow(den, -1, n) % n)
    return lam

# --- Repartidor: clave privada d y polinomio de grado k-1 sobre Z_n
k, m = 3, 5
d = 1 + secrets.randbelow(n - 1)
coefs = [d] + [secrets.randbelow(n) for _ in range(k - 1)]
partes = {i: sum(a * pow(i, j, n) for j, a in enumerate(coefs)) % n
          for i in range(1, m + 1)}
compromisos = [a * G for a in coefs]          # Feldman: C_j = a_j G
Q = compromisos[0]                            # C_0 = d G es la clave pública

# --- Cada custodio verifica su parte sin conocer las demás (Feldman)
for i, y in partes.items():
```

```
    lado_der = compromisos[0]
    for j in range(1, k):
        lado_der = lado_der + pow(i, j, n) * compromisos[j]
    assert y * G == lado_der, f"parte {i} inválida"

# --- Lagrange "en el exponente": Q se obtiene de las Q_i = y_i G
S = [2, 4, 5]
lam = lagrange0(S, n)
Qs = [partes[i] * G for i in S]               # claves públicas parciales
Q_rec = lam[0] * Qs[0] + lam[1] * Qs[1] + lam[2] * Qs[2]
assert Q_rec == Q                             # ¡sin reconstruir d!

# --- Reconstrucción de d y firma ECDSA
d_rec = sum(l * partes[i] for l, i in zip(lam, S)) % n
firma = SigningKey.from_secret_exponent(d_rec, curve=SECP256k1).sign(b"tx")
vk = VerifyingKey.from_public_point(Q, curve=SECP256k1)
assert vk.verify(firma, b"tx")
```

9.3 Firma de Schnorr de umbral: la clave nunca se reconstruye

Reconstruir d para firmar reintroduce el punto único de falla en el momento más delicado. En una firma de Schnorr, sin embargo, la parte secreta de la firma es *lineal* en d:

$$R = rG, \qquad c = H(R \,\|\, Q \,\|\, m), \qquad z = r + c\,d \pmod{q},$$

y se verifica comprobando $zG = R + cQ$. Si cada firmante i de un conjunto T de tamaño k elige su propio nonce r_i y calcula

$$z_i = r_i + c\,\lambda_i\,y_i \pmod{q}, \tag{5}$$

entonces $z = \sum_{i\in T} z_i = \sum r_i + c\sum \lambda_i y_i = r + c\,d$, con $R = \sum_{i\in T} r_i G$. La firma (R, z) es una firma de Schnorr ordinaria, verificable con Q, y d jamás existió completo. El siguiente programa, continuación del anterior, lo comprueba.

Listing 8: Firma de Schnorr de umbral didáctica (continúa `pk_secp256k1.py`)

```
import hashlib, secrets
from pk_secp256k1 import G, n, lagrange0, partes, Q   # reutiliza el reparto

def H(*datos):
    h = hashlib.sha256(b"".join(datos)).digest()
    return int.from_bytes(h, "big") % n

def ser(P):                                   # punto -> bytes
    return P.x().to_bytes(32, "big") + P.y().to_bytes(32, "big")

msg = b"autorizo la transferencia"
S = [1, 3, 4]                                 # firmantes presentes
lam = dict(zip(S, lagrange0(S, n)))

# Ronda 1: cada firmante elige un nonce y publica R_i = r_i G
r = {i: secrets.randbelow(n) for i in S}
R = r[S[0]] * G
for i in S[1:]:
    R = R + r[i] * G
```

```
20
21 # Ronda 2: reto común y firmas parciales z_i = r_i + c * lambda_i * y_i
22 c = H(ser(R), ser(Q), msg)
23 z = sum((r[i] + c * lam[i] * partes[i]) % n for i in S) % n
24
25 # Cualquiera verifica con la clave pública Q:  z G = R + c Q
26 assert z * G == R + c * Q
```

Sólo con fines didácticos

Este protocolo ilustra la idea, pero *no es seguro* tal cual: un firmante malicioso que elige su R_i después de ver los de los demás puede montar ataques de firmas concurrentes y falsificar firmas. FROST [6] añade compromisos previos de los nonces y "factores de vinculación" que impiden esos ataques; además, con un generador distribuido de claves ni siquiera el repartidor llega a conocer d. Para producción deben usarse implementaciones revisadas, como las de la Zcash Foundation [11]. Para ECDSA, cuya ecuación de firma no es lineal en d, los protocolos de umbral son considerablemente más complejos.

9.4 RSA: el patrón híbrido

Una clave privada RSA no vive en un campo primo. El exponente privado d satisface $ed \equiv 1 \pmod{\varphi(N)}$, y $\mathbb{Z}_{\varphi(N)}$ no es un campo: $\varphi(N)$ es par, de modo que las diferencias $x_j - x_m$ pares no son invertibles y la fórmula de Lagrange falla. Además, revelar $\varphi(N)$ a los custodios permitiría factorizar N. Shoup [8] resolvió el problema para firmas RSA de umbral multiplicando por $\Delta = n!$ para eliminar los denominadores, pero para el caso común de *respaldo* hay una solución mucho más simple:

1. cifrar la clave privada (por ejemplo, el archivo PEM) con AES en modo autenticado bajo una clave aleatoria de 128 bits;
2. repartir con Shamir *sólo* esa clave AES;
3. guardar el PEM cifrado en cualquier lugar, incluso en varios: sin la clave AES no revela nada.

Es el mismo diseño de la raíz de DNSSEC y el que recomienda la documentación de PyCryptodome. La etiqueta de autenticación de AES-GCM detecta, además, si alguno de los custodios presentó una parte alterada.

Listing 9: Respaldo de una clave RSA-3072: cifrado híbrido y reparto de la clave AES

```
1 from cryptography.hazmat.primitives.asymmetric import rsa, padding
2 from cryptography.hazmat.primitives import hashes, serialization as ser
3 from Crypto.Cipher import AES
4 from Crypto.Random import get_random_bytes
5 from Crypto.Protocol.SecretSharing import Shamir
6
7 # 1. Clave RSA y su serialización PEM (unos 2.5 KB: no cabe en una parte)
8 priv = rsa.generate_private_key(public_exponent=65537, key_size=3072)
9 pem = priv.private_bytes(ser.Encoding.PEM, ser.PrivateFormat.PKCS8,
10                          ser.NoEncryption())
11
12 # 2. Cifrado autenticado del PEM con una clave AES-128 aleatoria
13 kek = get_random_bytes(16)
14 c = AES.new(kek, AES.MODE_GCM)
15 ct, tag = c.encrypt_and_digest(pem)
```

```
paquete = c.nonce + tag + ct          # puede publicarse o respaldarse

# 3. Sólo la clave AES se reparte entre los custodios
partes = Shamir.split(3, 5, kek)
del kek, pem, priv

# 4. Recuperación con tres custodios
kek2 = Shamir.combine([partes[0], partes[2], partes[3]])
nonce, tag, ct = paquete[:16], paquete[16:32], paquete[32:]
pem2 = AES.new(kek2, AES.MODE_GCM, nonce=nonce).decrypt_and_verify(ct, tag)
priv2 = ser.load_pem_private_key(pem2, password=None)
```

9.5 Distribuir las partes cifradas con la clave pública de cada custodio

Hay un último cruce entre ambos mundos, ahora en sentido inverso: la criptografía de clave pública sirve para *entregar* las partes. Si cada custodio i tiene un par de claves (sk_i, pk_i), el repartidor cifra la parte y_i con pk_i y puede publicar o almacenar los n criptogramas en un solo lugar; sólo el custodio i puede descifrar la suya. Es lo que hace Vault con `-pgp-keys`, y combinado con los compromisos de Feldman permite que cada custodio verifique su parte al descifrarla. Esta combinación (reparto cifrado más verificación pública) es la base de la *compartición públicamente verificable* y de los generadores distribuidos de claves.

Guía rápida para claves de clave pública

- **Sólo respaldo, cualquier tipo de clave:** cifrar la clave con AES-GCM y repartir la clave AES (Sección 9.4).
- **Clave de curva elíptica con custodios que deben verificar:** repartir el escalar en $\mathbb{F}_q$ con compromisos de Feldman (Sección 9.2).
- **La clave no debe existir nunca completa:** firmas de umbral (FROST para Schnorr y Ed25519) con generación distribuida de claves.
- **Semilla de billetera:** SLIP-39.

10 Limitaciones y extensiones

El esquema básico es perfecto frente a un adversario *pasivo* con menos de k partes, pero no resuelve todo:

1. **Partes falsas o corruptas.** Un participante malicioso puede entregar una parte alterada y la reconstrucción producirá un valor incorrecto sin señal de error; peor aún, conociendo las demás partes, puede elegir qué valor falso se obtiene (Tompa y Woll [3]). Mitigaciones: autenticar el secreto (por ejemplo, con la etiqueta de AES-GCM o un hash publicado), usar sumas de verificación como SLIP-39, o reconstruir con más de k partes y comprobar consistencia.

2. **Repartidor deshonesto.** Los participantes no pueden verificar que sus partes provienen de un mismo polinomio de grado $k-1$. Los esquemas de *compartición verificable* (VSS) resuelven esto publicando compromisos de los coeficientes: Feldman [4] usa g^{a_j} (seguridad computacional) y Pedersen [5] usa compromisos $g^{a_j}h^{b_j}$ que ocultan la información de manera incondicional.

3. **Punto único al reconstruir.** Al recombinar, el secreto vuelve a existir completo en una máquina. Cuando el secreto es una clave de firma, la alternativa moderna son las *firmas de*

umbral (por ejemplo FROST, RFC 9591 [6]), en las que cada participante firma con su parte y la clave nunca se reconstruye (Sección 9.3).

4. **Exposición a largo plazo.** Un adversario puede ir obteniendo partes a lo largo de años. La compartición proactiva [7] renueva periódicamente las partes (ver la propiedad de renovación en la Sección 3) sin cambiar el secreto.

5. **Canales laterales.** Las implementaciones en Python puro no son de tiempo constante. Para claves de alto valor conviene una biblioteca en C auditada y un entorno controlado (módulo de seguridad en hardware o máquina aislada).

Resumen práctico

- Clave simétrica o archivo: PyCryptodome (clave de 16 bytes + AES-GCM).
- Semilla de billetera: `shamir-mnemonic` (SLIP-39).
- Clase o experimentos de cómputo multipartito: `shamirs` o la implementación de referencia de la Sección 6.
- Clave privada de curva elíptica: reparto en $\mathbb{F}_q$ con verificación de Feldman (Sección 9.2).
- Clave RSA o cualquier archivo de clave: patrón híbrido (Sección 9.4).
- Repartidor no confiable: VSS de Feldman o Pedersen.
- Clave de firma que no debe reconstruirse: firmas de umbral.

Referencias

[1] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[2] G. R. Blakley, "Safeguarding cryptographic keys," en *Proceedings of the National Computer Conference*, AFIPS, vol. 48, pp. 313–317, 1979.

[3] M. Tompa y H. Woll, "How to share a secret with cheaters," *Journal of Cryptology*, vol. 1, no. 2, pp. 133–138, 1988.

[4] P. Feldman, "A practical scheme for non-interactive verifiable secret sharing," en *Proc. 28th IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 427–438, 1987.

[5] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," en *Advances in Cryptology – CRYPTO '91*, LNCS 576, pp. 129–140, Springer, 1992.

[6] D. Connolly, C. Komlo, I. Goldberg y C. A. Wood, "The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures," RFC 9591, IETF, 2024.

[7] A. Herzberg, S. Jarecki, H. Krawczyk y M. Yung, "Proactive secret sharing or: How to cope with perpetual leakage," en *Advances in Cryptology – CRYPTO '95*, LNCS 963, pp. 339–352, Springer, 1995.

[8] V. Shoup, "Practical threshold signatures," en *Advances in Cryptology – EUROCRYPT 2000*, LNCS 1807, pp. 207–220, Springer, 2000.

[9] S. Josefsson e I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, IETF, 2017.

[10] NIST, "NIST First Call for Multi-Party Threshold Schemes," NIST IR 8214C, 20 de enero de 2026. https://csrc.nist.gov/pubs/ir/8214/c/final

[11] Zcash Foundation, `frost`: implementaciones en Rust de FROST (crates `frost-ed25519`, `frost-secp256k1`, entre otros). https://github.com/ZcashFoundation/frost

[12] Root Zone KSK Operator, *Root Zone KSK Operator Key Management Policy*, v3.6, IANA/PTI. https://www.iana.org/dnssec/procedures/ksk-operator/KSK_Key_Management_Policy_v3.6.pdf

[13] B. Schneier, "DNSSEC root key split among seven people," *Schneier on Security*, julio de 2010. https://www.schneier.com/blog/archives/2010/07/dnssec_root_key.html

[14] HashiCorp, "Seal/Unseal," documentación de Vault. https://developer.hashicorp.com/vault/docs/concepts/seal

[15] HashiCorp, "operator init," documentación de Vault. https://developer.hashicorp.com/vault/docs/commands/operator/init

[16] Ledger, "Part 1: Genesis of Ledger Recover – Self custody without compromise," 2023; *white paper* del protocolo en https://github.com/LedgerHQ/recover-whitepaper

[17] SatoshiLabs, "SLIP-0039: Shamir's Secret-Sharing for Mnemonic Codes." https://github.com/satoshilabs/slips/blob/master/slip-0039.md

[18] Trezor, `python-shamir-mnemonic`: implementación de referencia de SLIP-39. https://github.com/trezor/python-shamir-mnemonic

[19] PyCryptodome, "Secret Sharing Schemes." https://pycryptodome.readthedocs.io/en/latest/src/protocol/ss.html

[20] `shamirs`: Minimal pure-Python implementation of Shamir's Secret Sharing scheme. https://shamirs.readthedocs.io